

06 · Hardware-Beschleunigung (DMA)

Mikrocomputertechnik 3 · Direct Memory Access — die CPU vom Daten-Schaukeln befreien

Prof. Dr.-Ing. Daniel Klünder

Folie

1 / 75

Willkommen zu Block 6!

- Willkommen zur sechsten Vorlesung in Mikrocomputertechnik 3
- Bisher: Denken (Speicher/Caches), Reagieren (Interrupts), Handeln (PWM), Sprechen (Busse)
- Heute: Effizienz — den größten Stau im Datenpfad auflösen
- Ziel: stupide Fleißarbeit von der CPU auf Hardware-Assistenten verlagern

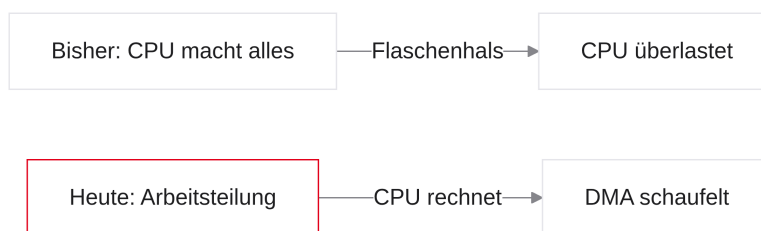
Notizen

Herzlich willkommen zurück! Sensoren und Aktoren beherrschen Sie bereits. Bisher erledigt die CPU jeden Schritt selbst. Heute lernen wir zu delegieren: Die CPU wird Manager, Subsysteme erledigen das Daten-Schaukeln.

Folie

2 / 75

Einordnung im Semester



Notizen

Von „alles selbst“ zu Parallelität: CPU rechnet, DMA bewegt Daten.

Anknüpfung: I2C-Labor

Kurz-Recap aus Block 5 (Vertiefung im Übungsblatt):

- IMU per I2C: Start, Adresse, R/W, ACK/NACK, Stop
- High/Low-Byte per Shift zu 16-Bit-Werten
- CPU wartete oft auf die I2C-Hardware (`wait_for_ack` / Polling)

```
uint8_t read_sensor_x(void) {  
    i2c_send_start();  
    /* ... */  
    return i2c_read_byte(); /* CPU wartet hier oft */  
}
```

Notizen

Für seltene Sensorwerte ist Polling akzeptabel. Bei großen, kontinuierlichen Datenmengen wird Wartezeit zum Problem.

Schnelle Schnittstellen, langsames Füttern

- Big 3 beherrscht: UART, SPI, I2C
- Externe Geräte anschließbar
- SPI für Displays/Flash oft mit SCK im Zehner-MHz-Bereich
- Folge: die Schnittstelle kann Megabytes/s transportieren — wenn Software nachkommt

Notizen

Schnelle Peripherie nützt wenig, wenn der Pfad RAM → Controller der Engpass ist.

Das Problem: Datenmenge

- Temperaturfühler: wenige Bytes/s — unkritisch
- Audio: z. B. $\approx 44\,100$ Samples/s, kontinuierlich, jitterarm
- TFT 320×240 , 16 Bit, 60 FPS:

$$320 \times 240 \times 2 \times 60 = 9\,216\,000 \text{ Bytes/s}$$

- CPU müsste millionenfach RAM lesen und SPI-Register füllen

Notizen

Bei Grafik/Audio bindet Software-Kopieren die CPU stark — selbst bei schneller Peripherie.

Refresher: Wie die CPU kopiert

- MCT1: Speicherzugriff über Load/Store
- `lw`: RAM → Register; `sw`: Register → MMIO (z. B. SPI-TX)
- Kein „direkt von RAM nach SPI“ in einem Befehl

```
loop:
    lw    t0, 0(s1)      /* RAM lesen */
    sw    t0, 0(s2)      /* Peripherie schreiben */
    addi  s1, s1, 4
    bne   s1, s3, loop
```

Notizen

Jedes Byte wandert durch Register/ALU-Pfad — viele Takte nur für 1:1-Kopieren.

Agenda

1. CPU-Flaschenhals: Polling und Interrupt-Stürme
2. DMA: Bypass der CPU und Bus-Mastering
3. Kanäle und Konfiguration (Source, Destination, Increment)
4. Burst, Ringpuffer, Double Buffering
5. Labor-Ausblick (Renode)

Notizen

Zuerst zeigen wir, warum Software scheitert — dann der Hardware-Rettungsanker DMA.

Lernziele Block 6

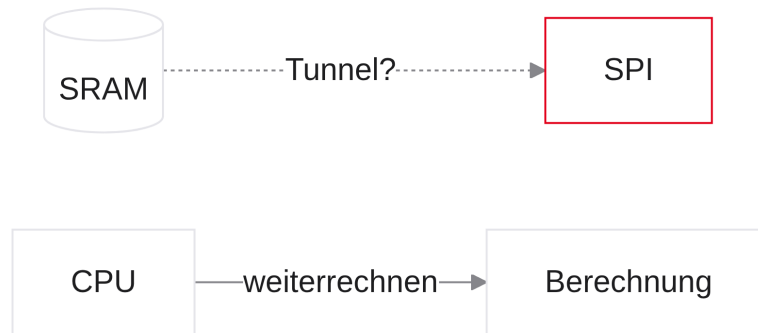
- Begründen, warum Polling und häufige ISRs den Prozessor auslasten
- Bus-Mastering und Arbitration im SoC erklären
- DMA-Kanal: Source, Destination, Increment Mode
- Cache-Kohärenz bei DMA einordnen
- Ringpuffer und Double Buffering (Ping-Pong) für Streams erklären

Notizen

Am Ende: große Transfers nicht mehr per `for`-Schleife über den Bus — sondern per DMA.

Die Leitfrage des Tages

- 1 MB Array im SRAM (z. B. Bild) → SPI → Display
- Parallel: CPU führt schwere Berechnung aus (z. B. Edge-AI)
- Wie fließen die Bytes RAM → Peripherie, ohne die CPU-Rechnung zu stoppen?



Notizen

Ziel: Parallelität — CPU rechnet, Datenfluss läuft daneben. Spoiler: DMA.

Folie

10/75

Fragen? — Motivation

- Limitierung durch `lw/sw` klar?
- Warum kann SPI „schneller“ sein als Software nachfüttert?

Notizen

Als Nächstes: klassische Software-Lösungen und warum sie bei 1 MB scheitern.

Folie

11/75

Methode 1: Polling

```
void send_bild_polling(uint8_t *bild, uint32_t length) {  
    for (uint32_t i = 0; i < length; i++) {  
        while ((SPI_STATUS_REG & TX_EMPTY_FLAG) == 0) { }  
        SPI_DATA_REG = bild[i];  
    }  
}
```

- Warten, bis TX leer — dann nächstes Byte
- Funktioniert korrekt — blockiert die CPU

Notizen

Klassischer Anfänger-Code: zuverlässig, aber die CPU dreht in der Warteschleife.

Die Polling-Falle

- Beispiel: SPI mit 10 MHz-SCK → grob $\approx 0,8 \mu\text{s}$ pro Byte (8 Bit + Overhead je nach Mode)
- 10^6 Bytes $\Rightarrow$ Größenordnung $\approx 0,8$ s Transferzeit
- CPU-Last in der `while`-Schleife: praktisch 100 % Wartezeit
- Keine parallele Nutzarbeit in dieser Zeit

Notizen

Sekundenbruchteile sind für GHz-CPU's Ewigkeiten — Milliarden verschwendeter Takte als „Postbote“.

Methode 2: Interrupt-Versand

- SPI löst bei TX-leer Interrupt aus (über PLIC)
- ISR schreibt ein Byte; CPU rechnet dazwischen weiter

```
volatile uint32_t bytes_sent;

void spi_tx_empty_isr(void) {
    if (bytes_sent < 1000000) {
        SPI_DATA_REG = bild_array[bytes_sent++];
    }
}
```

Notizen

Interrupts entlasten gegenüber Polling — bei Mikrosekunden-Raten droht trotzdem Kollaps.

Interrupt-Overhead

Pro Interrupt typisch:

- Pipeline leeren, `mepc/mcause` setzen
- Register retten (Stack)
- Nutzarbeit: ein Byte kopieren
- Register wiederherstellen, `mret`

Oft Dutzende Takte Overhead für wenige Nutz-Takte.

Notizen

Erinnerung Block 3: Kontextwechsel ist teuer — besonders bei einem Byte Nutzlast.

Interrupt Storm

- SPI meldet „leer“ im Sub- μ s- bis μ s-Takt
- Millionen ISRs/s möglich
- CPU: Zeit geht in Push/Pop statt Nutzarbeit

Phase	Was passiert
kurz	KI/Main
lang	ISR-Overhead
kurz	KI/Main
lang	ISR-Overhead

Notizen

Interrupts passen zu seltenen Ereignissen (Taste, Tick) — nicht zu Dauerfeuer im Mikrosekunden-Takt.

CPU als Postbote

- Polling und ISR: jedes Byte durch CPU-Register
- Pfad: RAM → Bus → (Cache) → Register → Bus → SPI
- ALU/Pipeline involviert, obwohl nur 1:1 kopiert wird

Notizen

Hochkomplexe Rechenwerke als Spediteur missbrauchen — architektonisch unnötig.

Bus-Engstelle beim Software-Kopieren

- Auch mit getrennten L1-I/D-Caches teilen sich CPU und Peripherie oft den Systembus zum Hauptspeicher
- Intensives lw/sw -Kopieren belegt diesen Pfad
- Instruction-Fetch und Nutzarbeit konkurrieren → Pipeline stottert

Notizen

Klassisch als Von-Neumann-Flaschenhals diskutiert; bei modernen SoCs oft die gemeinsame Memory-Interconnect. Software-Kopieren belegt denselben teuren Pfad.

Lösung: Hardware-Assistent

- CPU = Manager (rechnen, entscheiden)
- Auftrag: Quelle, Ziel, Länge — dann weiterarbeiten
- Assistent schaufelt und meldet „fertig“
- Name: Direct Memory Access (DMA) / DMAC

Notizen

Einmal konfigurieren, Hintergrundtransfer, Interrupt erst am Ende (oder bei Meilensteinen).

Fragen? — Flaschenhals

- Warum Interrupt-Sturm bei schnellem SPI?
- Warum verschwenden $1w/sw$ -Schleifen ALU/Bus?
- „Postboten“-Problem klar?

Notizen

Als Nächstes: Was DMA ist und wie Bus-Mastering funktioniert.

Was ist DMA?

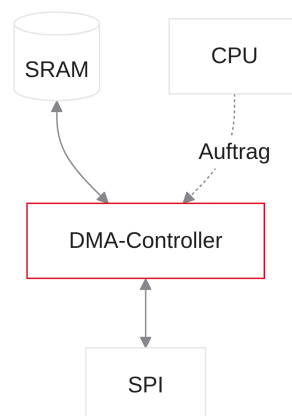
- DMA: Direct Memory Access — direkter Speicherzugriff
- Peripherie (SPI, UART, ADC, ...) liest/schreibt RAM ohne CPU-Kopierschleife
- Steuerung: DMA-Controller (DMAC) als dediziertes Subsystem

Notizen

Geräte umgehen die CPU und greifen — über den DMAC — direkt auf den Speicher zu.

Bypass der CPU

- Ohne DMA: RAM → CPU → UART/SPI
- Mit DMA: RAM → DMAC → Peripherie
- Datenpfad an der CPU vorbei; CPU-Auslastung für den Transfer nahe 0 %



Notizen

CPU erteilt den Auftrag und rechnet weiter; der Tunnel RAM–Peripherie läuft autonom.

Folie

22 / 75

Der DMA-Controller (DMAC)

- Eigenes Silizium-Modul auf dem SoC
- Keine allgemeine ALU — spezialisiert auf Adressen und Transfers
- Merkt sich u. a. Quelle, Ziel, Restlänge
- Oft mehrere Controller/Kanäle (DMA1, DMA2, ...)

Notizen

„Dumm, aber schnell“: lesen, schreiben, Zeiger erhöhen, N -mal — besser als jede Software-Schleife.

Folie

23 / 75

Kampf um den Bus

- Oft ein gemeinsamer Pfad zum RAM
- Früher: CPU alleiniger Nutzer
- DMA will denselben Bus — Konfliktregelung nötig

Notizen

Zwei Master am gleichen Medium brauchen Arbitration.

Folie

24 / 75

Bus-Mastering

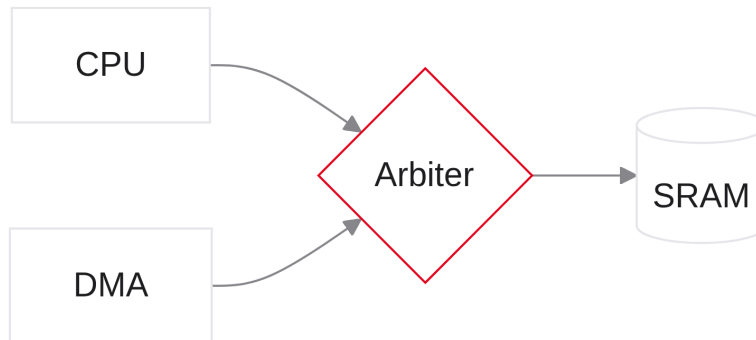
- Bus-Master: darf selbst Transaktionen starten
- CPU: Bus-Master
- DMAC: ebenfalls Bus-Master
- UART/SPI-Datenregister typisch: Slave (passiv)

Notizen

Die CPU verliert das Monopol; der DMAC darf eigenständig Speicherzugriffe auslösen.

Bus-Arbitration

- Bus-Matrix / Arbiter steuert den Zugang
- Gleichzeitiger Zugriff CPU und DMA → Priorität entscheidet
- Oft DMA bevorzugt (Echtzeit: Audio, Display)
- Verlierer: kurze Stall-Zyklen



Notizen

Echtzeit-Streams vertragen Stalls schlecht; die CPU verkraftet kurze Wartezeiten besser.

Cycle Stealing

- DMA „klaut“ gelegentlich Bus-Zyklen
- Kein ISR-Overhead pro Byte → Störung oft gering
- Mit L1-Cache rechnet die CPU oft ohne Dauerzugriff auf den Hauptspeicherbus

Notizen

Gegenüber Software-Kopieren vernachlässigbar — besonders bei guter Cache-Lokalität der CPU-Arbeit.

Transparent vs. Blocking

- Transparent: DMA nutzt Bus, wenn die CPU ihn ohnehin nicht braucht
- Blocking / Burst: DMA belegt den Bus länger — CPU stottert kurz
- Immer noch weit günstiger als C-Kopierschleifen

Notizen

Selbst harte Bursts sind Fliegengewicht gegen millionenfaches $1w/sw$.

Hardware-Handshake

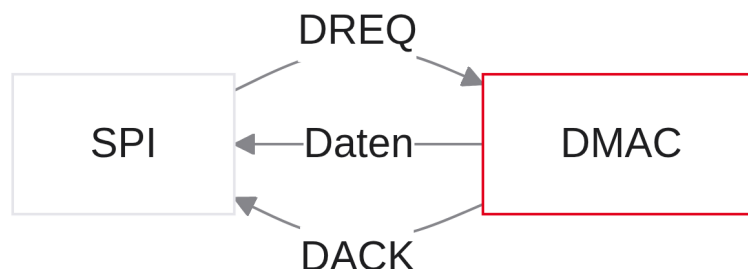
- DMA darf Peripherie nicht überfluten
- Synchronisation über dedizierte Signale im SoC (ohne CPU)

Notizen

Wie Staffelübergabe: erst anfordern, dann liefern.

Request / Acknowledge (Handshake)

- Request (klassisch oft DREQ): Peripherie „Register leer / bereit“
- DMAC: Transfer RAM → Peripherie
- Acknowledge (klassisch oft DACK): „geliefert“
- Signale und Namen sind herstellerspezifisch — das Prinzip bleibt gleich



Notizen

Klassische Bezeichnungen DREQ/DACK; am Chip oft andere Signalnamen. Prinzip: Peripherie fordert an, DMA liefert.

Folie

30 / 75

Richtung: Memory → Peripheral

- SRAM → UART / SPI / I2C / DAC
- Quelle: Array (Adresse steigt); Ziel: festes Datenregister
- Typisch: Audio-Out, Display-Refresh

Notizen

Klassischer Sender: C-Puffer hinaus in die Welt.

Folie

31 / 75

Richtung: Peripheral → Memory

- UART / SPI / ADC → SRAM
- z. B. ADC sampelt Mikrofon mit hoher Rate → DREQ → DMA füllt Puffer
- CPU wertet später aus (Spracherkennung etc.)

Notizen

Sensorik ohne Interrupt-Sturm: DMA füllt den Puffer geräuschlos.

Folie

32 / 75

Richtung: Memory → Memory

- SRAM-Array A → Array B
- Ersetzt/beschleunigt `memcpy`-artige Kopien
- Oft ohne externes DREQ — so schnell der Bus erlaubt

Notizen

Auch reine Software-Pufferumzüge profitieren vom DMAC.

DMA-Interrupts

- Anbindung an PLIC / Interrupt-System
- Typisch: Transfer Complete (TC), Half Transfer (HT), Transfer Error (TE)
- Ein Interrupt nach N Bytes statt N Interrupts

Notizen

HT ist Basis für Ping-Pong: eine Hälfte füllen/lesen, während die andere transferiert.

Fragen? — DMA-Grundlagen

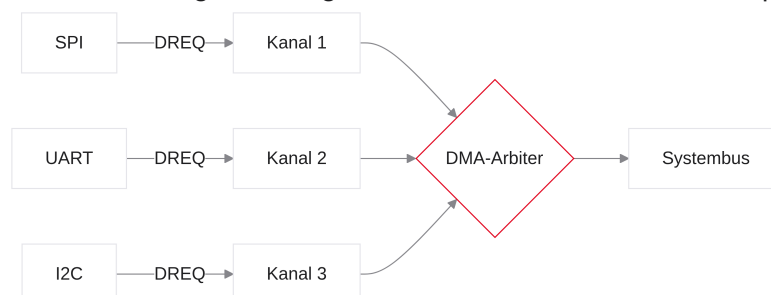
- Was bedeutet Bus-Mastering?
- Wie schlichtet der Arbiter (Cycle Stealing)?
- Rolle des Hardware-Handshakes (Request/Acknowledge)?

Notizen

Als Nächstes: Kanäle konfigurieren — Source, Destination, Increment, Cache.

DMA-Kanäle (Channels / Streams)

- Viele Peripherien wollen parallel Daten bewegen (UART, SPI, I2C, ...)
- Ein DMAC hat daher mehrere Kanäle (z. B. Channel 0–7)
- Jeder Kanal: eigene Quelle, Ziel, Länge, Priorität
- Interner DMA-Arbiter: bei gleichzeitigen DREQs entscheidet die Kanalpriorität



Notizen

Mehrere Förderbänder parallel: Audio auf Kanal 1, Logs auf Kanal 2 — der DMA-interne Schiedsrichter regelt Konflikte, ohne die CPU.

Folie

36 / 75

Multiplexing der Kanäle (DMAMUX)

- Oft mehr Peripherien als DMA-Kanäle
- Lösung: Multiplexer-Matrix (häufig DMAMUX) vor dem DMAC
- Software verbindet z. B. UART_TX-Request mit Kanal 3
- Doppelbelegung desselben Kanals → undokumentiertes / fehlerhaftes Verhalten
- Referenzhandbuch: Request-Tabellen sind Pflichtlektüre

Notizen

Erster Programmierschritt: Request-Leitung der richtigen Peripherie auf den richtigen Kanal legen.

Folie

37 / 75

MMIO-Register eines Kanals

- Steuerung über Memory-Mapped Register
- In C oft als `struct` abgebildet
- Vier Kernfragen: Woher? Wohin? Wie viel? Welche Breite / Modi?

```
typedef struct {
    volatile uint32_t SRC_ADDR;    /* Quelle */
    volatile uint32_t DST_ADDR;    /* Ziel */
    volatile uint32_t COUNT;       /* Anzahl */
    volatile uint32_t CONTROL;     /* Breite, Enable, ... */
} DMA_Channel_TypeDef;

/* fiktiv - Labor: Datenblatt / Renode */
#define DMA_CH1 ((DMA_Channel_TypeDef *) 0x40020000u)
```

Notizen

Layout und Bitfelder sind herstellerspezifisch — die vier Parameter finden Sie überall wieder.

Parameter 1: Source Address (SRC)

- Physikalische Startadresse der Quelle in SRC_ADDR
- Senden: oft Adresse eines Arrays im SRAM
- DMA liest von dort (ggf. mit Auto-Inkrement)

```
uint8_t bild_array[1000];  
DMA_CH1->SRC_ADDR = (uint32_t)&bild_array[0];
```

Notizen

Lade-Ort: Read-Zugriff auf dem Systembus in den internen DMA-Pfad.

Parameter 2: Destination Address (DST)

- Zieladresse in DST_ADDR
- Senden über SPI/UART: Adresse des Datenregisters (z. B. TX_DATA)
- DMA schreibt blind auf diese Adresse

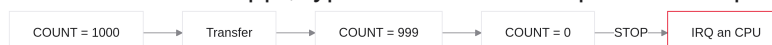
```
#define SPI_TX_DATA_REG 0x4003000Cu /* fiktiv */  
  
DMA_CH1->DST_ADDR = SPI_TX_DATA_REG;
```

Notizen

Ablade-Ort: nach dem Lesen folgt der Write auf die Peripherie — der Controller startet den seriellen Versand.

Parameter 3: Transfer Count

- COUNT: Anzahl der Transfers (oft 16 Bit, z. B. bis 65535)
- Nach erfolgreichem Transfer: Hardware dekrementiert automatisch
- Bei COUNT = 0: Kanal stoppt, typisch Transfer-Complete-Interrupt



Notizen

Ohne Zähler würde der DMA endlos weiterlaufen. Null bedeutet: Auftrag erledigt, CPU benachrichtigen.

Folie

41 / 75

Parameter 4: Data Width

- Pro Transfer: 8, 16 oder 32 Bit — gesetzt im CONTROL-Register
- Quelle und Ziel müssen passen (z. B. 8-Bit-Sensorregister $\neq$ 32-Bit-Write)

C-Typ	DMA-Breite	Transfers für 32 Bit Nutzdaten
uint8_t	Byte (8)	4
uint16_t	Halfword (16)	2
uint32_t	Word (32)	1 (oft maximal effizient)

Notizen

Passende Breite spart Bus-Zyklen: 16-Bit-Sensor $\rightarrow$ Halfword-DMA, nicht byteweise.

Folie

42 / 75

Address Increment Mode

- Arrays: Pointer muss wandern — sonst 1000 $\times$ dieselbe Adresse
- INC: nach jedem Transfer + Data-Width auf SRC und/oder DST
- Hardware-Äquivalent zu `pointer++`
- Quelle und Ziel getrennt schaltbar

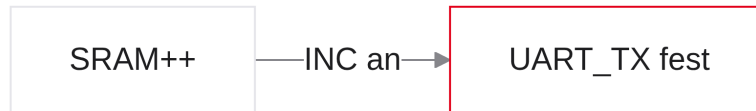
```
/* Konzept - in Hardware als Auto-Inkrement */
*DST_ADDR = *SRC_ADDR;
SRC_ADDR += width; /* wenn Memory-Increment aktiv */
```

Notizen

Damit wird die klassische Kopierschleife in Silizium nachgebaut.

Memory → Peripheral: Pointer

- Source Increment: **an** (Array durchlaufen)
- Destination Increment: **aus** (festes TX-Register)
- Ziel inkrementieren → Schreiben neben das Register → oft Bus Fault



Notizen

Klassischer Fehler: Peripherie-Adresse mitwandern lassen. Der Briefschlitz bleibt starr.

Peripheral → Memory: Pointer

- Source Increment: **aus** (festes ADC-/RX-Register)
- Destination Increment: **an** (Werte im Array ablegen)



Notizen

Sensor immer aus demselben Fach; Ablage im RAM der Reihe nach.

Memory Alignment

- Word-Transfers (32 Bit): Startadressen typisch 4-Byte-aligned
- Unaligned (z. B. 0x20000001) → oft Bus Error / Transfer Error
- In C Alignment erzwingen:

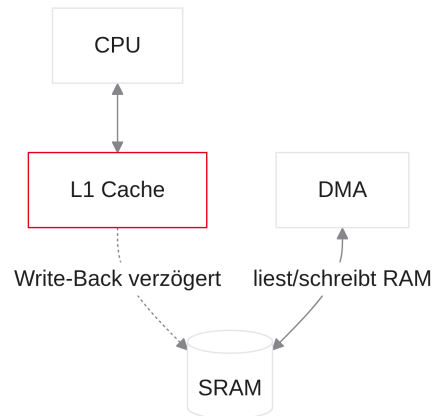
```
__attribute__((aligned(4))) uint8_t dma_buffer[1000];
```

Notizen

Unaligned Word-Zugriffe wären teuer oder verboten — DMA-Puffer sauber ausrichten.

Cache-Kohärenz und DMA

- Relevant, sobald ein Data-Cache (Write-Back) vorhanden ist — viele einfache MCUs haben keinen
- Erinnerung Block 2: L1-D-Cache puffert RAM-Inhalte
- DMA sieht den CPU-D-Cache typischerweise **nicht** — Zugriff auf den physischen Speicher
- CPU und DMA können unterschiedliche Sicht auf dieselben Adressen haben



Notizen

Klassische Falle: CPU ändert Array nur im Cache; DMA verschickt veraltete RAM-Bytes.

Stale Data (beide Richtungen)

- Senden (CPU → DMA): frische Daten nur im Cache → DMA sendet alte RAM-Inhalte
- Empfangen (DMA → CPU): frische Daten im RAM → CPU trifft veralteten Cache-Hit
- Lösung: Software-Cache-Wartung oder uncached Regionen

Notizen

Beide Seiten können sich „belügen“, wenn Kohärenz nicht hergestellt wird.

Cache Clean / Invalidate

- Clean (Flush): vor DMA-Sendestart schmutzige Lines des Puffers in den RAM schreiben
- Invalidate: vor CPU-Lesen empfangener DMA-Daten Lines verwerfen → nächster Read geht zum RAM
- API ist plattformabhängig (ARM: Cache-Maintenance; RISC-V: Fence ordnet Zugriffe; für Cache-Kohärenz ggf. Zicbom — `cbo.clean/cbo.inval`)

```
bild_berechnen();
/* Konzept: Cache-Bereich in den RAM schreiben */
cache_clean((uintptr_t)bild_array, sizeof(bild_array));
dma_start();
```

Notizen

Wenige Takte Overhead — dafür korrekte Sichtbarkeit zwischen CPU und DMA.

Uncached DMA-Zonen

- Alternative: Speicherregion ohne Caching (oft per MPU / PMP)
- CPU greift dort immer direkt auf RAM zu
- Kohärenz hardwareseitig einfacher; Zugriff etwas langsamer
- Linker platziert DMA-Puffer gezielt

```
MEMORY {
    RAM_CACHED      : ORIGIN = 0x20000000, LENGTH = 246K
    RAM_UNCACHED    : ORIGIN = 0x2003D800, LENGTH = 10K /* DMA */
}
```

Notizen

Professioneller Ansatz in OS/RTOS: DMA-Buffer in uncached Segment, kein manuelles Flushen vergessen.

Fragen? — Kanalkonfiguration

- Welche vier Parameter braucht ein DMA-Kanal?
- Warum Destination-Increment bei UART-TX aus?
- Cache-Kohärenz: Problem und Rolle von Clean/Invalidate?

Notizen

Als Nächstes: Burst vs. Single, Ringpuffer und Ping-Pong für endlose Streams.

Single-Transfer vs. Burst

- Single: ein DREQ → typisch ein Read + ein Write; Bus danach wieder frei
- Burst: nach einem Request ein Block (z. B. 16 Words) am Stück
- Burst: höherer Durchsatz, kurz härtere Busbelegung für die CPU

Notizen

Single ist höflich; Burst maximiert Bandbreite, blockiert den Bus länger.

Warum Burst-Transfers?

- On-Chip-SRAM: Burst spart vor allem wiederholte Bus-Arbitration
- Externes DRAM (Block 1): Zeile öffnen kostet; offene Zeile → schnelle Column-Zugriffe
- Burst: einmal „aufmachen“, viele Words hintereinander — besonders bei DRAM deutlich effizienter

Modus	Typischer Nachteil Single	Vorteil Burst
On-Chip-SRAM	16× Arbitration	1× Arbitration, hoher Durchsatz
DRAM	zusätzlich oft 16× Row-Aktivierung	1× öffnen, 16× lesen

Notizen

SRAM-MCU: Burst = weniger Bus-Schiedsverfahren. DRAM-SoC: zusätzlich die Zeilenlogik aus Block 1.

FIFOs im DMAC

- Problem: Burst holt schnell aus dem RAM; SPI/UART sind langsamer
- Viele DMACs: interner FIFO als Zwischenpuffer
- Burst → FIFO füllen → Bus freigeben → Peripherie per DREQ aus dem FIFO füttern



Notizen

Entkopplung: schneller Speicherzugriff und langsamer I/O-Takt getrennt.

Problem endloser Streams

- Audio-DAC: z. B. $\approx 44\,100$ Samples/s, lange Dauer
- RAM fasst nicht die ganze Datei
- Kleiner Puffer: $\text{COUNT} = 0 \rightarrow$ Sound bricht ab
- Ständiges Stoppen/Neustarten in Software $\rightarrow$ Glitches

Notizen

Feste Blöcke reichen für Streams nicht — der DMA muss „im Kreis“ weiterlaufen können.

Circular Mode (Ringpuffer)

- Array fester Größe als Ring (z. B. 4 KiB)
- Bei $\text{COUNT} = 0$: Pointer und Zähler werden in Hardware neu geladen (Wrap)
- DMA rotiert, bis Software den Kanal stoppt

Notizen

Industriestandard für Audio, Video, Netzwerk: Speicher als Endlosschleife.

Ringpuffer: Software-Pflicht

- DMA-Lesezeiger läuft im Kreis
- CPU muss den gerade verlassenenen Bereich nachfüllen (z. B. aus Flash/Netz)
- Zu langsam → alte Daten → Stottern (Buffer Underrun)



Notizen

Katz-und-Maus: DMA frisst, Software füllt — wo steht der DMA gerade?

Double Buffering (Ping-Pong)

- Ring in zwei Hälften A und B
- DMA liest A → CPU füllt parallel B (und umgekehrt)
- Kein Konflikt: getrennte Bereiche

Notizen

Zwei Eimer: einer wird geleert, der andere gefüllt — stabile Multimedia-Wiedergabe.

Ping-Pong: Interrupt-Logik

- Mitte erreicht → Signal: Hälfte A fertig → CPU füllt A nach
- Ende erreicht → Signal: Hälfte B fertig → Wrap, CPU füllt B nach
- Endlos mit klarer Phasengrenze

Notizen

Zwei Ereignisse reichen: Halb und Voll (Complete) im Circular Mode.

Half-Transfer-Interrupt (HT)

- Bei vielen Controllern: HT bei exakt 50 % von COUNT → erste Hälfte nachfüllen
- TC (COUNT = 0): zweite Hälfte nachfüllen (bei Circular: Wrap)
- Ein Array + zwei Flags in der ISR (Namen/Bits herstellerspezifisch)

```
void dma_isr(void) {  
    if (STATUS_REG & HALF_TRANSFER_FLAG)  
        fill_first_half();  
    if (STATUS_REG & TRANSFER_COMPLETE_FLAG)  
        fill_second_half();  
}
```

Notizen

HT + TC = Double Buffering ohne zwei separate Hardware-Kanäle.

Ausblick: DMA2D (2D-Kopie)

- High-End-SoCs: 2D-DMA / Grafik-Accelerator (z. B. Chrom-ART)
- Versteht Rechtecke (X/Y), nicht nur lineare Bytes
- Zeilenoffsets am Ende jeder Zeile — Ausschnitte aus Framebuffer-Layouts

Notizen

Für GUIs: Button aus Hintergrundbild kopieren, ohne CPU-Pixelschleifen.

Scatter-Gather (verkettete Listen)

- Viele kleine Puffer im RAM verstreut
- Neu-Konfigurieren nach jedem Puffer kostet CPU/IRQs
- Scatter-Gather: Deskriptor-Liste (SRC, DST, COUNT, Next) im RAM
- DMA lädt den nächsten Auftrag selbst nach

Notizen

Königsklasse: CPU schreibt die Auftragskette; DMA arbeitet sie autonom ab.

Folie

62 / 75

Grenzen von DMA

- Verschiebt Bytes — keine allgemeine Logik
- Kein `if` auf Nutzdaten, keine Berechnung pro Byte
- Kein Protokollverständnis (z. B. I2C-ACK): nur Reaktion auf DREQ/Handshake
- Für Entscheidungen bleibt die CPU zuständig

Notizen

Assistent ohne Gehirn: transportiert zuverlässig; bewertet und filtert nicht.

Folie

63 / 75

Fragen? — Streams und Burst

- Warum ist Burst (SRAM: Arbitration; DRAM: zusätzlich Row-Hits) effizienter?
- Was leistet Circular Mode?
- Ping-Pong mit Half-Transfer: Ablauf?

Notizen

Theorie Streams abgeschlossen. Als Nächstes: Labor-Ausblick — Polling vs. DMA messbar machen.

Folie

64 / 75

Ausblick Labor: DMA in Renode

Geplantes Szenario (Details im Übungsblatt):

- Langer String (z. B. HTML-Header) über UART
- Runde 1: Polling — CPU blockiert sichtbar
- Runde 2: DMA-Kanal — CPU rechnet/blinkt weiter, Text läuft im Hintergrund

Notizen

Dieselben Dämonen wie in der Theorie: erst blockierend, dann asynchron entlastet.

Messung 1: Polling (blockierend)

- LED als Heartbeat der `main`-Schleife
- Klassische UART-Sendeschleife:

```
void send_uart_polling(const char *text, uint32_t len) {  
    for (uint32_t i = 0; i < len; i++) {  
        while (!(UART_STATUS_REG & TX_EMPTY)) { }  
        UART_TX_REG = text[i];  
    }  
}
```

Notizen

Vergleichswert: die C-Schleife, die wir theoretisch als Engpass begründet haben.

Erwartung: Timer-/LED-Beweis

- Während langer Polling-Ausgabe: Text im UART-Monitor
- LED steht (nahezu) still — Heartbeat pausiert
- Interpretation: Polling blockiert Multitasking in der Hauptschleife

Notizen

Laboridee: Solange gesendet wird, kommt die `main`-Schleife nicht zum Blinken.

Messung 2: Umstieg auf DMA

- Polling-Schleife ersetzen durch MMIO-Konfiguration des DMAC
- Generisches Rezept in vier Schritten (herstellerübergreifend)

Notizen

Registerlayout im Labor aus Datenblatt / Renode-Modell — Bits hier didaktisch.

Schritt 1: UART für DMA scharfschalten

- Standard: UART erwartet CPU-Writes
- Enable-Bit für DMA-Request (DREQ), wenn TX-Register Platz hat

```
#define UART_CR3      (*(volatile uint32_t *) (UART_BASE + 0x08u))
#define UART_DMA_EN   (1u << 7) /* Bitlage fiktiv */

UART_CR3 |= UART_DMA_EN;
```

Notizen

Ohne Request vom UART startet der DMA nicht — Peripherie muss „rufen“ dürfen.

Schritt 2: Kanal konfigurieren

- SRC = String im SRAM, DST = UART-TX-Register (fest)
- COUNT = Länge; Memory-Increment an, Peripherie-Increment aus

```
DMA_CH1->SRC_ADDR = (uint32_t)html_string;
DMA_CH1->DST_ADDR = (uint32_t)&UART_TX_REG;
DMA_CH1->COUNT    = string_laenge;
DMA_CH1->CONTROL   |= (DMA_MINC_EN | DMA_DIR_M2P);
```

Notizen

Exakt die Parameter aus dem Theorieteil — Array wandert, TX-Adresse bleibt starr.

Schritt 3: Interrupts aktivieren

- Transfer-Complete-Enable im DMA
- PLIC (bzw. Interrupt-Controller) für den Kanal freischalten

```
DMA_CH1->CONTROL |= DMA_TCIE_EN;
enable_interrupt(DMA_CH1_IRQn); /* Pseudocode */
```

Notizen

Ein IRQ am Ende statt eines IRQs pro Byte — Abschluss und ggf. nächster Auftrag.

Folie

71 / 75

Schritt 4: Enable (Startschuss)

- ENABLE-Bit setzen → Transfer startet hardwareseitig
- C-Funktion kehrt sofort zurück — CPU ist frei

```
void start_dma_transfer(void) {  
    DMA_CH1->CONTROL |= DMA_ENABLE_BIT;  
    /* main() läuft sofort weiter */  
}
```

Notizen

Ein Bit, und der Hintergrundtransfer läuft — Triumph der Delegation.

Folie

72 / 75

Erwartung mit DMA

- Text erscheint weiter im Monitor
- LED blinkt ungestört (Heartbeat bleibt)
- CPU kann parallel rechnen; DMA nutzt Cycle Stealing

Notizen

Zielbild im Simulator: System „lebt“, während Bytes asynchron ausgehen.

Folie

73 / 75

Interrupt-Handling am Ende

- COUNT = 0 → Transfer Complete
- In der ISR Status quittieren (Flag clear), sonst erneutes Feuern

```
void dma_channel1_isr(void) {
    if (DMA_STATUS_REG & TRANSFER_COMPLETE) {
        DMA_CLEAR_REG = CLEAR_TRANSFER_COMPLETE;
        /* optional: nächsten Transfer starten */
    }
}
```

Notizen

Wie beim SysTick: Grund lesen, Flag löschen, zurück — Mission erfüllt.

Folie

74 / 75

Zusammenfassung Block 6

- Polling blockiert; Interrupt-Stürme kosten Overhead
- DMA: spezialisierter Coprozessor für Speichertransfers
- DMAC als Bus-Master, Cycle Stealing mit dem Arbiter
- Kanal: SRC, DST, COUNT, Breite, Increment
- Cache-Kohärenz: Clean/Invalidate oder uncached Regionen
- Circular Mode, HT/TC → Ringpuffer und Ping-Pong

Notizen

CPU bleibt Gehirn; DMA ist der Muskel für Massenbewegungen.

Folie

75 / 75

Ausblick und Abschluss

- Übungsblatt: Polling vs. DMA in Renode nachvollziehen
- Nächster Block: moderne CPU-Beschleunigung (u. a. Out-of-Order)

Vielen Dank — viel Erfolg im Labor.

Notizen

Wenn Terminal flüssig schreibt und die LED weiterblinkt, ist die Architektur verstanden. Bis zum nächsten Block!